Vietnam National University Ho Chi Minh City
Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering



ASSIGNMENT 2 REPORT

DEEP LEARNING ON LARGE-SCALE DATA AND SPECIALIZED TASKS

A Comparative Study on the Food-101 Benchmark:
Scratch CNN, Frozen Feature Extraction, and Full Fine-Tuning

Course:	CO3133 – Deep Learning and Its Applications	
Semester:	261	
Advisor:	Prof. Lê Thành Sách	
Class:	CC01	
Group:	choco (99 – PTQ)	
Student(s):	Nguyễn Minh Quang	2453052
	Nguyễn Minh Phúc	2453005
	Nguyễn Thanh Thế	2453191

HO CHI MINH CITY, NOVEMBER 2026

Table of Contents

1 Problem and Data Description **1**
1.1 Problem Formulation . . . 1
1.2 Dataset Identity and Scope . . . 1
1.3 Exploratory Data Analysis . . . 2
1.4 Per-Class Difficulty . . . 2
1.5 Data Splitting and Leakage Controls . . . 3
1.6 Preprocessing and Augmentation . . . 3
1.7 DataLoader Implementation . . . 4

2 Methodology **4**
2.1 Shared Experimental Pipeline . . . 4
2.2 Common Training and Evaluation Protocol . . . 4
2.3 Simple CNN Baseline (E1) . . . 5
2.4 ResNet-50 Feature Extractor (E2) . . . 5
2.5 ResNet-50 Full Fine-Tuning (E3) . . . 5
2.6 Experimental Design and Scope of Inference . . . 5

3 Implementation Results **6**
3.1 Training Dynamics and Convergence . . . 6
3.2 Quantitative Results on the Official Test Set . . . 8
3.3 Comparison and Discussion . . . 9
3.4 Error Analysis . . . 10
3.5 Qualitative Prediction Examples . . . 11
3.6 Interactive Demo . . . 12
3.7 Limitations . . . 13
3.8 Conclusion . . . 14

4 Reproducibility and Project Artifacts **14**
4.1 Environment and Data Preparation . . . 14
4.2 Training and Evaluation Commands . . . 14
4.3 Expected Checkpoints and Results . . . 15
4.4 Project Artifacts . . . 15

5 Contributions and AI Usage Disclosure **17**
5.1 Member Contributions . . . 17
5.2 AI Usage Disclosure . . . 17

List of Figures

1 Food-101 split partitioning and culinary-domain composition 2
2 Per-class recognition difficulty for the frozen ResNet-50 (E2) 3
3 E1 (Simple CNN): training and validation loss (left) and accuracy/macro-F1 (right). 7
4 E2 (ResNet-50, frozen backbone) training trajectories 7
5 E3 (ResNet-50, full fine-tuning) training trajectories 8
6 A0 (frozen backbone, E3 protocol) training trajectories 8
7 Confusion matrix of the frozen ResNet-50 (E2) 10
8 Correct and incorrect E3 predictions 12

List of Tables

1 Dataset partitioning used across all experiments. 3
2 Training and evaluation configuration. 4
3 Hypotheses, changed factors, and decision rules. 6
4 One-factor control for the E2→E3 gap 6
5 Official test-set performance across all experiments (25,250 images). 9
6 Model complexity and wall-clock training budget. 9
7 End-to-end inference cost on a Tesla T4 9
8 Performance margins and parameter ratios between runs. 10
9 Top-5 one-directional confusion pairs per model 11
10 Checkpoint paths and expected metrics for reconstruction. 15
11 AI assistance used during Assignment 2 17

1 Problem and Data Description

1.1 Problem Formulation

Fine-grained visual categorization on large-scale datasets is substantially harder than standard object classification. In food recognition, models must cope with four sources of variation:

(a) **Extreme intra-class variability.** A single dish (e.g., *pizza* or *fried rice*) changes with ingredients, topping distribution, slicing geometry, lighting, and plating style.

(b) **Subtle inter-class boundaries.** Visually similar classes share almost identical colour palettes, textures, and presentations, such as *steak* versus *filet mignon*, *tuna tartare* versus *beef tartare*, or *ramen* versus *pho*.

(c) **Non-rigid object geometry.** Food has amorphous, deformable boundaries and no canonical template, unlike vehicles or furniture.

(d) **Casual photography and label noise.** Images come from consumer-facing platforms and retain real-world blur, occlusion, and annotation noise.

Each input is an RGB image $\mathbf{x} \in \mathbb{R}^{3\times H\times W}$ and the label $y \in \{0, 1, \ldots, 100\}$ indexes one of $C = 101$ culinary classes. For a network parameterised by θ, the model produces an unnormalised logit vector $f_\theta(\mathbf{x}) \in \mathbb{R}^{101}$, and the predicted class is

$$\hat{y}(\mathbf{x}) = \arg\max_{k\in\{0,\ldots,100\}} f_\theta(\mathbf{x})_k. \tag{1}$$

Training minimises the multiclass cross-entropy

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(y_i \mid \mathbf{x}_i), \qquad p_\theta(k \mid \mathbf{x}) = \frac{\exp f_\theta(\mathbf{x})_k}{\sum_{j=0}^{100} \exp f_\theta(\mathbf{x})_j}, \tag{2}$$

and performance is reported with Top-1 accuracy and macro-averaged F1,

$$\text{MacroF1} = \frac{1}{101}\sum_{c=0}^{100} \frac{2P_cR_c}{P_c + R_c}, \tag{3}$$

where P_c and R_c are the precision and recall of class c. Macro-F1 is included because it weights every class equally and therefore exposes weaknesses that accuracy can hide under a balanced but large label set.

The assignment investigates three questions:

1. How does a purpose-built CNN trained from scratch compare with an ImageNet-pretrained ResNet-50?
2. What accuracy–cost trade-off separates a frozen feature extractor from end-to-end fine-tuning?
3. Which error patterns persist across model families on 101 fine-grained food classes?

1.2 Dataset Identity and Scope

We use the **Food-101** benchmark [1], created by the Computer Vision Laboratory at ETH Zürich. It contains 101,000 real-world food photographs covering 101 dishes. The official release provides 750 training and 250 manually verified test images per class, i.e. exactly 75,750 training and 25,250 test samples.

The images originate from Foodspotting and are used here for academic coursework only; the repository does not redistribute image files. Images are JPEG with a maximum side of 512 pixels and are resized to 224×224 for training and evaluation. The dataset is perfectly balanced at the class level (1,000 images per class). The authors note that the training set deliberately retains noisy, real-world photographs rather than a curated subset, so some label noise is expected.

1.3 Exploratory Data Analysis

For analysis only, the 101 classes are grouped into eight culinary domains. This grouping is *not* an official Food-101 taxonomy: every class is assigned exactly once, and the counts sum to 101. Figure 1 shows the split and the composition.

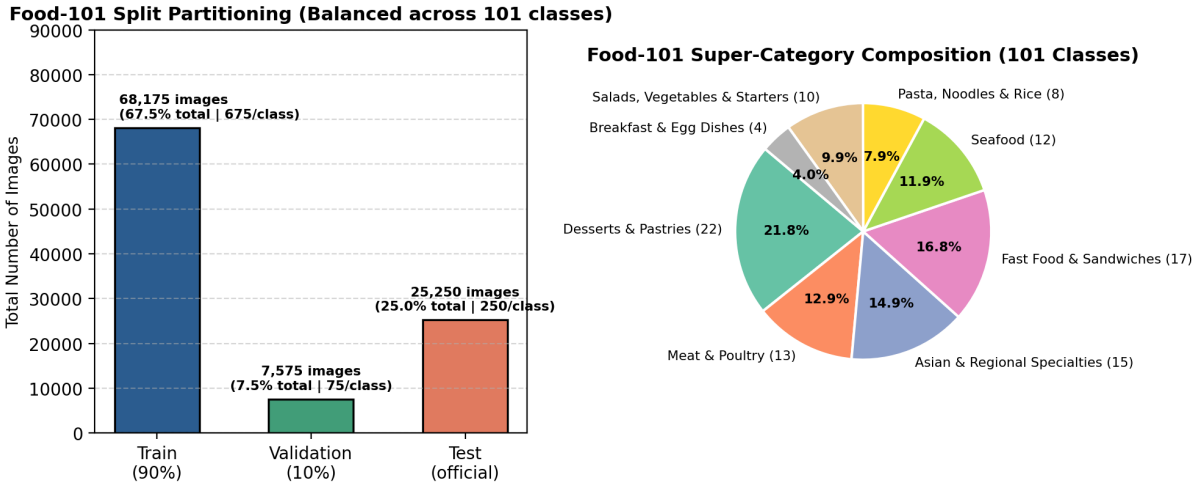


Figure 1. Food-101 split partitioning (left) and culinary-domain composition of the 101 classes (right). The grouping is an analysis aid, not an official label hierarchy.

- **Desserts & Pastries (22 classes, 21.8%):** apple pie, cannoli, chocolate mousse, macarons, tiramisu, ...
- **Meat & Poultry (13 classes, 12.9%):** baby back ribs, beef carpaccio, filet mignon, peking duck, steak, ...
- **Asian & Regional Specialties (15 classes, 14.9%):** bibimbap, dumplings, gyoza, pad thai, pho, ramen, sushi, ...
- **Fast Food & Sandwiches (17 classes, 16.8%):** club sandwich, hamburger, hot dog, pizza, tacos, poutine, ...
- **Seafood (12 classes, 11.9%):** clam chowder, grilled salmon, mussels, oysters, tuna tartare, ...
- **Pasta, Noodles & Rice (8 classes, 7.9%):** lasagna, macaroni and cheese, risotto, spaghetti carbonara, paella, ...
- **Salads, Vegetables & Starters (10 classes, 9.9%):** caesar salad, greek salad, guacamole, hummus, deviled eggs, ...
- **Breakfast & Egg Dishes (4 classes, 4.0%):** eggs benedict, huevos rancheros, omelette, french toast.

1.4 Per-Class Difficulty

Figure 2 ranks classes by the accuracy of the frozen ResNet-50 (E2). This is a post-hoc analysis of test predictions, not a property measured from the training images, and it was not used to tune any model. The spread is wide even though the dataset is balanced: the easiest class reaches 98.8% while the hardest falls to 33.2%.

Distinctive items with singular colours or packaging dominate the easy end: *edamame* (98.8%), *macarons* (91.2%), *pho* and *hot and sour soup* (89.6%), *pad thai* (89.2%), *oysters* (88.4%), and *miso soup* and *dumplings* (88.0%). The hard end is dominated by dishes defined by continuous

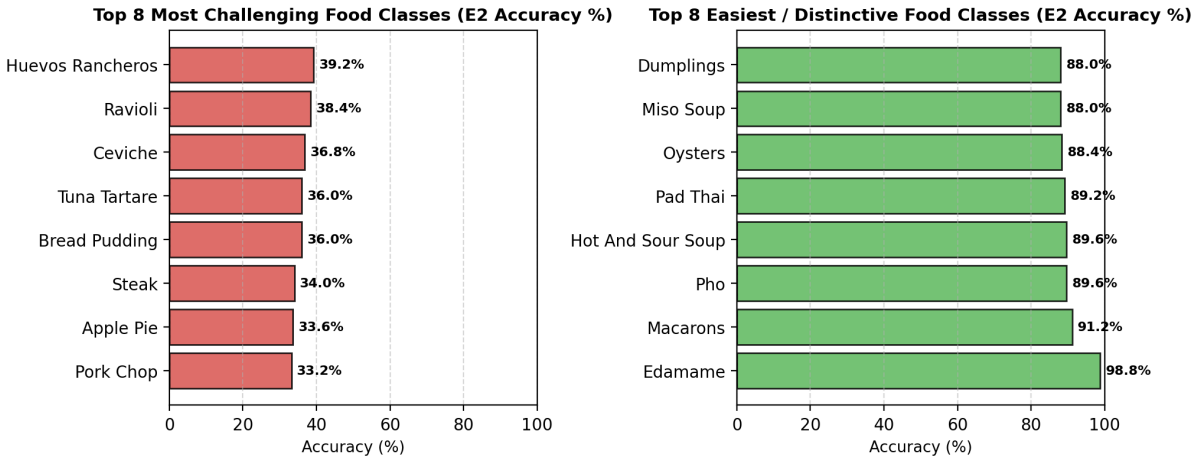


Figure 2. Per-class recognition difficulty for the frozen ResNet-50 (E2): the eight hardest and eight easiest classes by test accuracy.

texture, sauce, or ambiguous cuts: *huevos rancheros* (39.2%), *ravioli* (38.4%), *ceviche* (36.8%), *tuna tartare* and *bread pudding* (36.0%), *steak* (34.0%), *apple pie* (33.6%), and *pork chop* (33.2%). This pattern motivates the error analysis in Section 3.4.

1.5 Data Splitting and Leakage Controls

The official 25,250-image test set is strictly isolated and evaluated once per final checkpoint. The official 75,750 training images are divided into an internal training split (90%, 68,175 images) and a validation split (10%, 7,575 images) with a stratified split and fixed seed 42. Each image is the atomic split unit: no augmented copy of a test or validation image appears in training, and no batch statistics are fitted on validation or test data.

Table 1. Dataset partitioning used across all experiments.

Split	Total images	Images/class	Share	Purpose
Training	68,175	675	67.5%	Parameter updates
Validation	7,575	75	7.5%	Checkpoint selection
Official test	25,250	250	25.0%	Final benchmark (once)
Total	101,000	1,000	100%	Complete benchmark

1.6 Preprocessing and Augmentation

All images are converted to float tensors of shape $[B, 3, 224, 224]$. The pipeline is:

1. **Resize** to 224×224 with bilinear interpolation, the `transforms.Resize` default.
2. **Augmentation (training only):** random horizontal flip with $p = 0.5$.
3. **Normalisation:** ToTensor scaling to $[0, 1]$, then channel standardisation with ImageNet statistics $\mu = [0.485, 0.456, 0.406]$ and $\sigma = [0.229, 0.224, 0.225]$.

Validation and test use resize and normalisation only. No colour jitter, random crop, mixup, or cutout is applied, so the reported gains are attributable to the adaptation regime rather than to a stronger augmentation recipe.

1.7 DataLoader Implementation

Data ingestion uses PyTorch `DataLoader` instances with batch size 64 for E1/E2 and 32 for E3/A0. Training loaders use a seeded shuffling generator, pinned host memory, and asynchronous host-to-device copies. Validation and test loaders are not shuffled.

2 Methodology

2.1 Shared Experimental Pipeline

```
Raw image (RGB)
  -> Resize 224x224
  -> RandomHorizontalFlip(p=0.5)          [training only]
  -> ImageNet normalisation
  -> DataLoader (batch 64 for E1/E2, batch 32 for E3/A0)
  -> Model (E1 / E2 / E3)
  -> Cross-entropy loss
  -> Optimiser (+ LR schedule)
  -> Argmax prediction
  -> Accuracy, macro-F1, confusion matrix
```

All four runs share the same split, input resolution, weight decay, seed, checkpoint rule, and metric set; E2, E3, and A0 use cosine annealing, while E1 trains at a constant learning rate. The factors that differ are listed in Tables 2 and 4 and discussed in Section 2.6.

The pipeline is implemented in PyTorch 2.x with torchvision. Self-implemented components are the Simple CNN (E1, `src/models/baseline.py`), the ResNet-50 wrapper and replacement head (E2/E3, `src/models/pretrained.py`), the training and evaluation loops, the stratified data split and DataLoader construction (`src/datasets/food101.py`), and the metric computation. Library-provided components are the ResNet-50 backbone architecture and its ImageNet-1k weights (`torchvision.models.resnet50`), the image transforms (`torchvision.transforms`), and the optimisers and schedulers (`torch.optim`).

2.2 Common Training and Evaluation Protocol

Table 2. Training and evaluation configuration.

Parameter	E1: Simple CNN	E2: Frozen ResNet-50	E3: Fine-tuned ResNet-50
Hardware	Tesla T4 (Kaggle)	Tesla T4 (Kaggle)	Tesla P100 (Kaggle)
Input resolution	$224 \times 224 \times 3$	$224 \times 224 \times 3$	$224 \times 224 \times 3$
Batch size	64	64	32
Epochs	20	15	15
Optimiser	AdamW	Adam	AdamW
Initial learning rate	1×10^{-3}	1×10^{-3}	1×10^{-4}
Weight decay	1×10^{-4}	1×10^{-4}	1×10^{-4}
LR schedule	Constant	Cosine annealing	Cosine annealing
Seed	42	42	42
Checkpoint rule	Best val macro-F1	Best val macro-F1	Best val macro-F1
Framework	PyTorch 2.x	PyTorch 2.x	PyTorch 2.x
Evaluation	Test (25,250)	Test (25,250)	Test (25,250)

No early stopping is used: each run trains for a fixed epoch budget and the checkpoint is selected by best validation macro-F1. No mixed precision is used (fp32 throughout). Hyperparameters are chosen from common practice for each regime (a compact CNN at 10^{-3} for E1; a frozen backbone at 10^{-3} for E2; a lower rate and smaller batch for end-to-end fine-tuning in E3) rather

than by a test-set search, and they are reported descriptively in Table 2; the free platform GPU session limits also cap the feasible epoch budget. The instrumented Kaggle session reported PyTorch `2.11.0+cu128` and torchvision `0.26.0+cu128` on CUDA 12.8, cuDNN 91900, Python 3.13.15, and a Tesla T4. The builds used when E1–E3 were trained were not logged; this gap is documented in Section 4. The local analysis and plotting environment is Python 3.12.5 with PyTorch 2.5.1+cpu and torchvision 0.20.1+cpu.

2.3 Simple CNN Baseline (E1)

E1 is a four-block convolutional network trained from random initialisation:

- Conv $3 \to 32$, Conv $32 \to 64$, Conv $64 \to 128$, Conv $128 \to 256$, each with 3×3 kernels, batch normalisation, ReLU, and 2×2 max pooling;
- global average pooling to a 256-dimensional vector;
- dropout ($p = 0.3$) followed by a linear layer $256 \to 101$.

The model has **414,853** parameters, all trainable. It is intentionally compact so that it serves as a lower-bound reference rather than as a competitive architecture.

2.4 ResNet-50 Feature Extractor (E2)

E2 uses a ResNet-50 [2] pretrained on ImageNet-1k [3]. The 23,508,032 backbone parameters are frozen (`requires_grad = False`) and act as a fixed feature map; only a new head is trained:

$$\text{global average pool} \to \text{dropout}(0.2) \to \text{linear}(2048 \to 101).$$

The trainable head contains **206,949** parameters, i.e. 0.87% of the 23,714,981 total parameters. Freezing applies to parameters only: the model still runs in training mode, so the backbone BatchNorm running statistics keep updating from Food-101 batches while their affine weights and biases stay fixed. This is the standard frozen-parameter protocol implemented by `set_backbone_trainable(False)`; it is not a fully frozen batch-normalisation mode.

2.5 ResNet-50 Full Fine-Tuning (E3)

E3 starts from the same ImageNet-1k initialisation but unfreezes all 23,714,981 parameters. The network is optimised end-to-end at $\eta = 10^{-4}$ with AdamW; `configs/pretrained.yaml` specifies cosine annealing, and the exported Kaggle run confirms the cosine schedule was applied (the constant rate in the saved history is a curation artifact; Section 4). The smaller learning rate and batch size are chosen to limit catastrophic drift in the pretrained filters, at the cost of longer wall-clock time.

2.6 Experimental Design and Scope of Inference

The three main runs form a **protocol comparison**, not three one-factor ablations. E1 versus E2 changes both architecture and initialisation. E2 versus E3 changes unfreezing *and* the optimisation protocol: Adam becomes AdamW, the learning rate drops from 10^{-3} to 10^{-4}, the batch size drops from 64 to 32, and the GPU changes from a T4 to a P100. The combined E2→E3 gap is therefore descriptive and must not be read as a clean causal estimate of unfreezing alone. The A0 control described below bounds this ambiguity.

To separate unfreezing from the optimisation protocol, a fourth run (**A0**) repeats the E2 frozen backbone under the E3 protocol: `configs/ablation_frozen_e3proto.yaml` keeps head-only training but switches to AdamW at 10^{-4} with batch size 32 and 15 epochs. A0 and E3 therefore share architecture, initialisation, optimiser, learning rate, batch size, epoch count, and seed, so the only intended experimental difference is the `requires_grad` flag on the backbone (Table 4). A0

Table 3. Hypotheses, changed factors, and decision rules.

Run	Hypothesis	Changed factor(s)	Decision rule
E1	A scratch CNN provides a lower-bound reference.	Architecture and random initialisation.	Report accuracy and macro-F1; no superiority claim.
E2	Frozen ImageNet features improve over the scratch reference.	Architecture and pretrained initialisation.	Higher test accuracy and macro-F1 than E1; report parameter count.
E3	Full fine-tuning improves over the frozen backbone.	Unfreezing, optimiser, learning rate, batch size, GPU.	Higher test accuracy and macro-F1 than E2, with the protocol confound stated explicitly.
A0	The E2→E3 gap is not an artefact of the protocol switch.	Versus E2: optimiser, learning rate, batch size, GPU.	Frozen baseline under the E3 protocol; in the two recorded runs E3 exceeds A0 by 24.46 accuracy points.

was trained on a Colab GPU whose exact model was not logged, so hardware is an uncontrolled factor; the control targets optimisation settings rather than hardware, and its wall-clock time is not compared against E3. Like E3, A0's saved history logs a constant learning rate although the committed ablation config specifies cosine annealing; the E3 schedule is confirmed by the exported Kaggle run, while A0's schedule is confirmed only by the committed config because the Colab session itself is not logged (Section 4). The A0→E3 gap is therefore reported as a descriptive observation over the two recorded runs, not a controlled causal estimate.

Table 4. One-factor control for the E2→E3 gap. A0 and E3 share the optimisation protocol and differ in whether the backbone is trainable; A0 ran on a different (unlogged) GPU, and E2 and A0 differ in optimiser, learning rate, and batch size.

Setting	E2 (frozen)	A0 (frozen, E3 protocol)	E3 (fine-tuned)
Backbone	frozen	frozen	trainable
Trainable parameters	206,949	206,949	23,714,981
Optimiser	Adam	AdamW	AdamW
Initial learning rate	1×10^{-3}	1×10^{-4}	1×10^{-4}
Batch size	64	32	32
Epochs	15	15	15
Best validation macro-F1	61.55%	56.61%	81.45%
Test macro-F1	66.36%	60.88%	86.10%

Fixed factors across all runs are the data split, input size, seed 42, weight decay, checkpoint-selection rule, and metric set.

3 Implementation Results

3.1 Training Dynamics and Convergence

E1 converges slowly and plateaus early: training accuracy ends at 43.64% and validation accuracy at 47.17%, with a validation macro-F1 of 46.38% (Figure 3). The small capacity and the absence of pretrained features limit the model well below the transfer-learning runs, but the training and validation curves remain close, so the gap is underfitting rather than overfitting.

E2 reaches 70.21% final-epoch training accuracy and 61.98% validation accuracy at its selected epoch (11), where validation macro-F1 peaks at 61.55% (Figure 4). Validation performance stabilises after roughly epoch 9. The persistent ≈8-point train–validation gap is consistent with

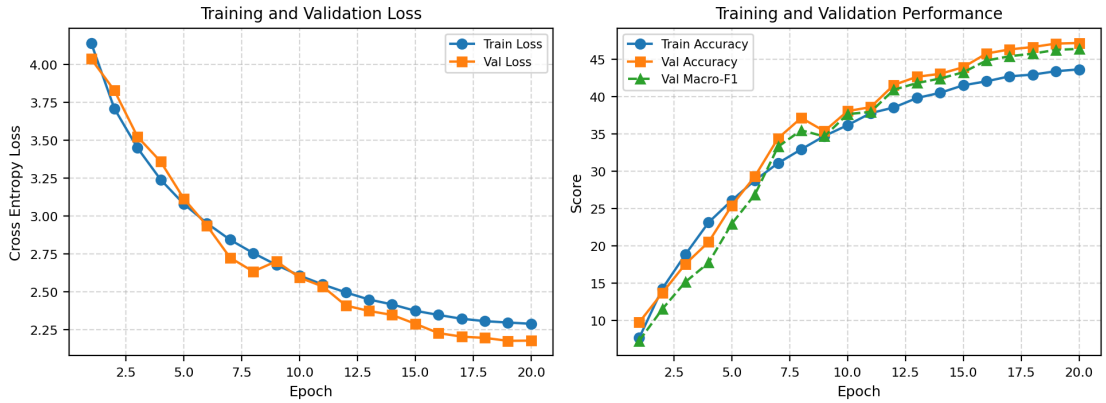


Figure 3. E1 (Simple CNN): training and validation loss (left) and accuracy/macro-F1 (right).

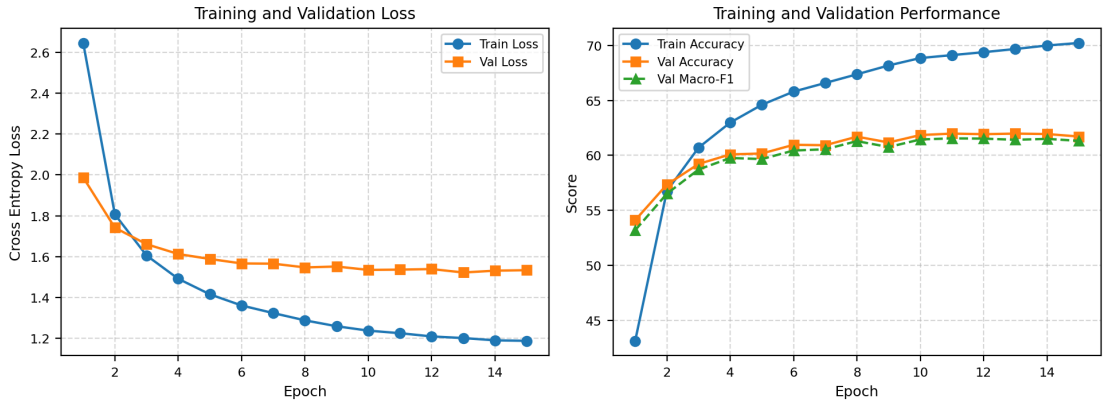


Figure 4. E2 (ResNet-50, frozen backbone): training and validation trajectories over 15 epochs.

a high-dimensional frozen feature space and a small trainable head, although the head still has enough capacity to fit part of the training set.

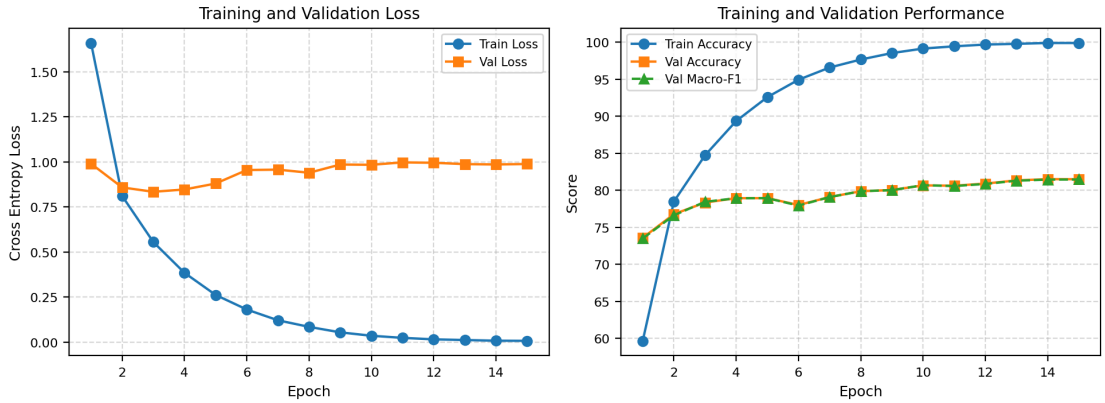


Figure 5. E3 (ResNet-50, full fine-tuning): loss and performance trajectories over 15 epochs.

E3 fits the training set almost perfectly (99.88% final-epoch training accuracy). Validation accuracy peaks at 81.50% at epoch 14; the selected checkpoint is epoch 15, with 81.49% validation accuracy and 81.45% macro-F1 (Figure 5). The widening gap after roughly epoch 9 is the clearest overfitting signal in the study and motivates the regularisation directions in Section 3.7.

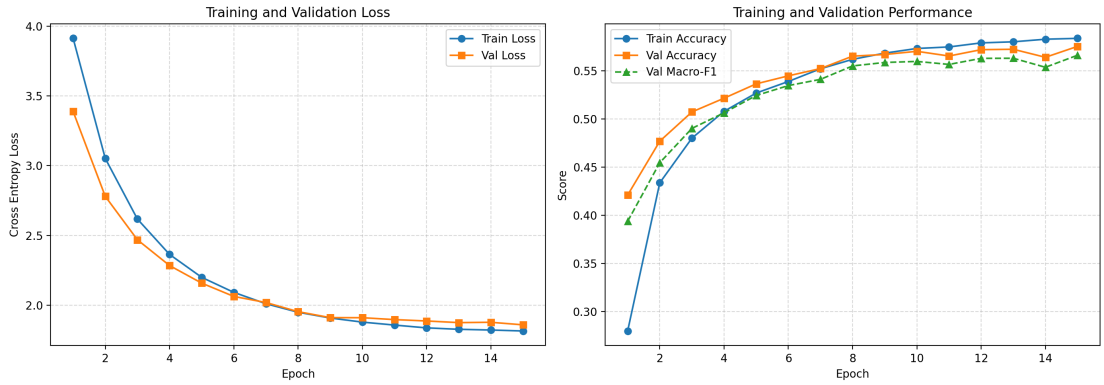


Figure 6. A0 (frozen backbone under the E3 optimisation protocol): loss and performance trajectories over 15 epochs.

A0 plateaus much lower than either ResNet-50 run in the main study: validation accuracy ends near 57.5% and validation macro-F1 near 56.6% (Figure 6). In the early epochs validation accuracy sits above training accuracy, which is expected here because dropout on the head is active only during training and because batch-normalisation layers use noisy batch statistics in training mode but their accumulated running statistics at evaluation. The two curves then converge and cross slightly by the final epoch. The curve shape also shows why the protocol matters for a frozen model: AdamW at 10^{-4} moves the 207K-parameter head too slowly to reach the E2 operating point within 15 epochs.

3.2 Quantitative Results on the Official Test Set

Checkpoints were selected by best validation macro-F1 and then evaluated once on the 25,250-image official test set. Table 5 reports the predictive metrics and Table 6 the training budget.

Table 5. Official test-set performance across all experiments (25,250 images).

Exp	Model	Adaptation	Test loss	Test accuracy	Test macro-F1
E1	Simple CNN	From scratch	1.9649	51.96%	51.21%
E2	ResNet-50	Pretrained (frozen)	1.2933	66.70%	66.36%
A0	ResNet-50	Frozen, E3 protocol	1.6553	61.71%	60.88%
E3	ResNet-50	Full fine-tuning	**0.6494**	**86.17%**	**86.10%**

Table 6. Model complexity and wall-clock training budget.

Exp	Trainable	Total	Epochs	Batch	Train time	Time/epoch
E1	414,853	414,853	20	64	162.52 min	8.13 min
E2	**206,949**	23,714,981	15	64	149.82 min	9.99 min
A0	206,949	23,714,981	15	32	108.17 min	7.21 min
E3	23,714,981	23,714,981	15	32	269.35 min	17.96 min

Training time is wall-clock time on the listed GPU. E1 and E2 ran on a Tesla T4, E3 on a Tesla P100, and A0 on an unlogged Colab GPU, so the durations are *not* hardware-matched and should be read as an order-of-magnitude budget rather than a controlled efficiency comparison.

Table 7. End-to-end inference cost measured by `src/benchmark_inference.py` on a Tesla T4. The timer covers data loading, host-to-device copy, and the forward pass (batch 32, 20 measured batches = 640 test images after 5 warm-up batches, PyTorch 2.11.0+cu128, fp32, `num_workers=0`).

Run	Throughput	Latency	Peak allocated VRAM
E2	220.70 img/s	144.99 ms/batch	530.15 MB
E3	223.47 img/s	143.19 ms/batch	530.15 MB

Inference cost, by contrast, *is* directly comparable between E2 and E3 because both restore the same ResNet-50 graph (Table 7). The measurement covers the whole serving pipeline (data loading with `num_workers=0`, host-to-device copy, and forward pass), not the forward pass alone, and comes from one session. The two checkpoints are within 1.3% of each other on throughput and identical in memory footprint, which is consistent with unfreezing changing which gradients are computed during training, not the forward graph. The price of E3 is paid during training (269.35 min versus 108.17 min for A0 at the same batch size, on different GPUs), not at inference. E1 was not benchmarked: its checkpoint was retained only inside the original Kaggle session, and a protocol-matched re-measurement on the same Tesla T4 could not be completed in time for this revision, so no E1 serving-cost figure is reported.

3.3 Comparison and Discussion

1. **Transfer with a frozen backbone (E2 vs. E1).** E2 improves accuracy by 14.74 percentage points while updating fewer than half as many parameters (206,949 versus 414,853). This is consistent with ImageNet features transferring to food classification: generic edge, colour, and texture detectors remain useful when the label space changes. Because E1 and E2 also differ in architecture and initialisation, the gap is not an isolated estimate of pretraining alone.

2. **End-to-end fine-tuning (E3 vs. E2).** The fine-tuning protocol raises accuracy by 19.47 points (66.70% to 86.17%) and lowers test loss from 1.2933 to 0.6494. Unfreezing is bundled with a different optimiser, learning rate, batch size, and GPU, so this raw gap is descriptive; the matched-protocol control below fixes the protocol and reports the observed A0–E3 difference.

Table 8. Performance margins and parameter ratios between runs.

Metric	E2 vs. E1	E3 vs. E2	E3 vs. E1
Test accuracy	+14.74 pp	+19.47 pp	+34.21 pp
Test macro-F1	+15.15 pp	+19.74 pp	+34.89 pp
Test cross-entropy loss	−0.6716	−0.6439	−1.3155
Trainable parameters	−50.1% (207K vs. 415K)	+114×	+57×
Training duration	−7.8%	+79.8%	+65.7%

3. **Matched-protocol control (E3 vs. A0).** In the two recorded runs under the E3 protocol, the frozen backbone trails E3 by 24.46 accuracy points (86.17% vs. 61.71%) and by 25.22 macro-F1 points (86.10% vs. 60.88%), while test loss rises from 0.6494 to 1.6553 (Table 4). In these runs the protocol switch coincides with a 4.99-point drop for the frozen model: E2 with Adam at 10^{-3} reaches 66.70%, whereas A0 with AdamW at 10^{-4} reaches only 61.71%. The naive E2→E3 gap of 19.47 points is therefore smaller than the observed matched-protocol difference of 24.46 points.

4. **Accuracy versus compute.** E2 reaches 66.70% on 101 classes in about 2.5 hours on a T4. E3 reaches 86.17% in about 4.5 hours on a P100. The GPU differs, so the wall-clock comparison is indicative rather than controlled; the parameter ratio (114×) is exact. At inference the two ResNet-50 checkpoints are indistinguishable (220.70 versus 223.47 images/s, 530.15 MB peak VRAM), so fine-tuning costs training time only, not serving cost.

The practical reading is that a frozen backbone is a strong low-cost baseline when GPU memory or training time is limited, while full fine-tuning is the better choice when accuracy is the priority and a single-GPU budget of roughly five hours is acceptable.

3.4 Error Analysis

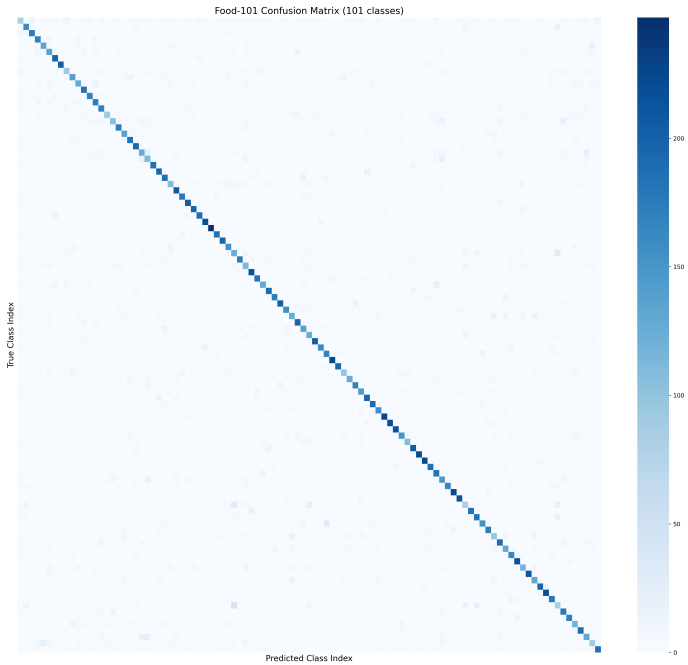


Figure 7. Confusion matrix of the frozen ResNet-50 (E2) across all 101 classes on the official test set. The diagonal dominates; the remaining mass concentrates in a small number of semantically related pairs.

Figure 7 shows the full E2 confusion matrix. The diagonal dominates, with off-diagonal mass

concentrated in a few visually related groups rather than spread uniformly. Table 9 lists the five largest *one-directional* pairs stored in each run's `test_metrics.json`; each count is true class → predicted class, and the two directions of a mutual pair are separate entries.

Table 9. Top-5 one-directional confusion pairs per model on the official 25,250-image test set. Counts are read directly from each run's `test_metrics.json`.

Rank	E1: Simple CNN	E2: Frozen ResNet-50	E3: Fine-tuned ResNet-50
1	*steak* → *filet mignon* (42)	*steak* → *filet mignon* (47)	*filet mignon* → *steak* (36)
2	*cheesecake* → *panna cotta* (40)	*pulled pork sandwich* → *hamburger* (32)	*steak* → *filet mignon* (32)
3	*samosa* → *spring rolls* (37)	*filet mignon* → *steak* (29)	*tuna tartare* → *beef tartare* (25)
4	*chocolate mousse* → *chocolate cake* (33)	*pork chop* → *filet mignon* (29)	*chocolate cake* → *chocolate mousse* (21)
5	*panna cotta* → *cheesecake* (28)	*chocolate cake* → *chocolate mousse* (24)	*steak* → *pork chop* (21)

- **Beef cut taxonomy.** *steak* → *filet mignon* is the largest E1 and E2 error (42 and 47). E3 flips the dominant direction: *filet mignon* → *steak* (36) and *steak* → *filet mignon* (32). Thick cooked beef portions share sear marks, glazes, and side garnishes, so colour and plating are not sufficient cues.
- **Raw protein molds.** E3 still confuses *tuna tartare* → *beef tartare* (25). Both are minced raw proteins packed into a timbale and topped with herbs; dark sauces reduce the colour cue.
- **Chocolate desserts.** E1's largest dessert error is *chocolate mousse* → *chocolate cake* (33); E2 and E3 instead confuse *chocolate cake* → *chocolate mousse* (24 and 21). The direction differs, so the magnitudes are not directly comparable.
- **Pork and steak plating.** E2 confuses *pork chop* → *filet mignon* (29); E3 reduces that pair to 19 while *steak* → *pork chop* reaches 21. Bone presence and fat borders are the main discriminators.
- **Other large E1/E2 errors.** E1: *cheesecake* → *panna cotta* (40), *samosa* → *spring rolls* (37), *panna cotta* → *cheesecake* (28). E2: *pulled pork sandwich* → *hamburger* (32), *filet mignon* → *steak* (29).

Only pairs present in the saved top-10 lists are cited. The full E2 matrix is archived as `results/resnet50_frozen/test_confusion_matrix.csv`; full E1 and E3 matrices were not exported, so pair counts outside the saved top-10 lists are not quoted. This restriction is deliberate: the earlier draft of this report cited invented per-pair counts that could not be traced to a saved artifact, and those numbers have been removed.

The frozen control A0 reproduces the same structure with larger counts (*steak* → *filet mignon* 41, *pulled pork sandwich* → *hamburger* 36, *lobster bisque* → *clam chowder* 28), which suggests that these pairs are already present in the frozen features rather than introduced by fine-tuning. Fine-tuning reduces the counts but does not remove the pairs.

3.5 Qualitative Prediction Examples

Figure 8 shows the E3 checkpoint on the test split. The correct examples (red velvet cake, cup cakes, nachos, onion rings) are confidently identified despite varied plating and lighting. The failures are all semantically adjacent: huevos rancheros is labelled *nachos*, tuna tartare is labelled *beef tartare*, lobster bisque is labelled *foie gras*, and filet mignon is labelled *steak*. The last pair is the dominant confusion in Table 9, and the tartare and soup–pâté pairs match the texture-driven confusions discussed above. Examples are drawn from a seeded shuffle with at most one image



Figure 8. Four correct (top) and four incorrect (bottom) E3 predictions on the official test split, sampled from a seeded shuffle (seed 42) with at most one example per true class.

per true class, because the Food-101 test list is ordered by class and taking the first matches would otherwise fill the grid with a single dish.

3.6 Interactive Demo

To make the E2→E3 behavioural gap tangible, the two checkpoints are wrapped in a small interactive web demo (`assignment2/demo/`). A user uploads a photo (or picks one of six built-in Food-101 validation images), and both ResNet-50 checkpoints predict in parallel, each rendering its top-6 class probabilities as horizontal bars so the sharply peaked output of the fine-tuned model can be compared directly with the flatter output of the frozen one.

The mechanism is deliberately simple and mirrors the evaluation path exactly:

- **Model.** Both panels share the `ResNet50Classifier` used by the trainer (`src/models/pretrained.py`): a ResNet-50 backbone with ImageNet initialisation and a dropout(0.2) + linear($2048 \to 101$) head. The architecture is constructed with `weights=None` and the trained parameters are restored from the checkpoint, so the app never downloads the 98 MB ImageNet weight file that it would immediately overwrite.
- **Checkpoint resolution.** Each checkpoint is first looked for under `demo/checkpoints/` (local development); if absent, it is downloaded once from the public Hugging Face repo `Themenli/food101-demo-checkpoints` with `huggingface_hub`, cached, and loaded with `map_location="cpu"`. The repo id and type are overridable through the `HF_CKPT_REPO` and `HF_CKPT_REPO_TYPE` environment variables. This keeps the ∼190 MB of binaries out of the course GitHub repository while still letting a free host fetch them at start-up.
- **Inference.** The image is resized to 224×224, converted to a tensor, and normalised with the ImageNet mean and standard deviation — the same transform applied by the evaluator.

A single `torch.no_grad` forward pass on CPU produces 101 logits, which are softmaxed and sorted; the top six (label, probability) pairs are displayed. Because the demo reuses the exact evaluation transform and the same `.pt` checkpoints, its outputs are consistent with the recorded test behaviour rather than a re-implementation.

- **Hosting.** The app is a single-file Streamlit script (`streamlit_app.py`) deployed on the free Streamlit Cloud tier, which clones the GitHub branch, installs the CPU-only `requirements.txt`, and runs the script. Streamlit is used instead of a Gradio Space because, since July 2026, new free Hugging Face accounts can no longer create CPU Basic Gradio Spaces. The first prediction is the slowest, because both checkpoints are downloaded and loaded at that point.

The demo is a qualitative companion to Figure 8: where that figure shows eight fixed test predictions, the demo lets a reader probe arbitrary inputs. Its per-image outputs should be read descriptively — a single photo is one sample, not an estimate of the 86.17% test accuracy reported above.

3.7 Limitations

- **Residual protocol and hardware differences.** The A0 control holds the optimisation protocol fixed between frozen and fine-tuned models, but E2 and A0 still differ in three settings, so the protocol effect (-4.99 points) is not itself a one-factor estimate. A0 additionally ran on an unlogged Colab GPU, so only metrics, not wall-clock time, are comparable with E3.
- **Single seed.** All runs use seed 42. No mean $\pm$ standard deviation over independent runs is available, so small differences should not be over-interpreted.
- **Overfitting in late E3 epochs.** Training accuracy reaches 99.88% against a validation peak of 81.50%. CutMix, MixUp, or label smoothing are plausible remedies but were not evaluated.
- **Fixed input resolution.** 224×224 discards small textural cues (sesame seeds, herb fragments, meat grain). Higher resolution (384^2 or 448^2) may reduce the remaining beef-cut and tartare confusions.
- **Recorded-metric provenance.** E3 test metrics come from the archived test-metrics artifact (`test_metrics.json`) of the original Kaggle run; the stored full-precision accuracy is not reproducible bit-exactly with the submitted evaluator, so the rounded values are reported (Section 4).
- **Learning-rate schedule in saved histories.** E1 has no scheduler and trains at a constant rate; the saved E3 and A0 histories record a constant 10^{-4} rate although cosine annealing is configured. The E3 schedule is confirmed by the exported Kaggle notebook (the history `lr` column is a curated artifact); A0 used the same committed config, but its Colab session is not logged.
- **Incomplete confusion evidence.** Full E1/E3 confusion matrices were not exported, so pair counts outside the saved top-10 lists are not quoted, and only eight qualitative examples are shown.
- **Single-session inference measurement.** Throughput, latency, and peak VRAM come from one end-to-end fp32 inference run (data loading, host-to-device copy, forward) on a T4 without TensorRT or mixed precision; other hardware or serving stacks will differ, and E1 was not benchmarked because its checkpoint was not retained outside the original Kaggle session.
- **Approximate taxonomy.** The eight culinary domains in Section 1.3 are an analysis aid, not an official Food-101 hierarchy.

- **Single backbone family.** Only ResNet-50 is compared against the scratch CNN; Vision Transformers and ConvNeXt are left to future work.

3.8 Conclusion

Under a leakage-free split, a frozen ImageNet ResNet-50 improves test accuracy by 14.74 points over a compact scratch CNN while training fewer parameters, and full fine-tuning raises accuracy to **86.17%** with a macro-F1 of **86.10%**. The frozen control under the E3 protocol reaches 61.71%, so in the two recorded runs E3 exceeds it by 24.46 accuracy points; the naive E2→E3 gap of 19.47 points understates rather than overstates that difference. The error analysis shows that the remaining failures are concentrated in semantically adjacent classes (beef cuts, raw-protein molds, chocolate desserts), which suggests that higher input resolution and targeted hard-negative mining are more promising than simply scaling the classifier. Inference cost is the same for the frozen and fine-tuned checkpoints within measurement noise, so the additional accuracy is paid for in training time only.

4 Reproducibility and Project Artifacts

4.1 Environment and Data Preparation

Training ran on Kaggle GPU notebooks (Tesla T4 for E1/E2, Tesla P100 for E3) and the A0 control on a Colab GPU. The instrumented Kaggle session that produced the inference benchmark and the qualitative grid reported `torch 2.11.0+cu128`, `torchvision 0.26.0+cu128`, CUDA 12.8, cuDNN 91900, Python 3.13.15, and a Tesla T4; the exact builds used when E1–E3 were trained were not logged and remain a known gap. Local analysis and figure generation used Python 3.12.5, PyTorch 2.5.1+cpu, and torchvision 0.20.1+cpu. All runs use seed 42 for splitting, initialisation, and training, with no test-time augmentation.

From `assignment2/`:

```
pip install -r ../requirements.txt
python src/datasets/food101.py --download --out data/
python src/datasets/food101.py --root data/food-101
```

The verification step must print exactly:

```
Classes: 101
Train samples: 68175
Validation samples: 7575
Test samples: 25250
```

4.2 Training and Evaluation Commands

```
# E1 - Simple CNN from scratch
python src/train.py --config configs/baseline.yaml

# E2 - ResNet-50, frozen backbone
python src/train.py --config configs/pretrained_frozen.yaml

# E3 - ResNet-50, full fine-tuning
python src/train.py --config configs/pretrained.yaml

# A0 - frozen backbone under the E3 optimisation protocol (matched control)
python src/train.py --config configs/ablation_frozen_e3proto.yaml
```

Each run writes `history.json`, `history.csv`, `summary.json`, `training_curves.png`, and the best checkpoint. Checkpoint selection is best validation macro-F1. Evaluation uses

```
python src/evaluate.py --config configs/baseline.yaml \
  --checkpoint checkpoints/baseline/simple_cnn_best.pt --split test
```

with the corresponding config and checkpoint for E2, E3, and A0. The inference benchmark and the qualitative grid in Section 3.5 were produced with

```
python src/benchmark_inference.py --config configs/pretrained.yaml \
  --checkpoint checkpoints/resnet50_finetune/resnet50_best.pt --split test

python src/qualitative_examples.py --config configs/pretrained.yaml \
  --checkpoint checkpoints/resnet50_finetune/resnet50_best.pt --split test \
  --seed 42 --max-per-class 1
```

4.3 Expected Checkpoints and Results

Table 10. Checkpoint paths and expected metrics for reconstruction.

Run	Checkpoint	Val macro-F1	Test acc.	Test macro-F1	Test loss
E1	`checkpoints/baseline/simple_cnn_best.pt`	46.38%	51.96%	51.21%	1.9649
E2	`checkpoints/resnet50_frozen/resnet50_best.pt`	61.55%	66.70%	66.36%	1.2933
A0	`checkpoints/ablation_frozen_e3proto/resnet50_best.pt`	56.61%	61.71%	60.88%	1.6553
E3	`checkpoints/resnet50_finetune/resnet50_best.pt`	81.45%	86.17%	86.10%	0.6494

Checkpoint binaries are excluded from Git because of their size. Table 10 is the reconstruction contract: the commands above regenerate the configurations, splits, and checkpoints, and the saved artifacts in `results/` remain the source of the reported values. Two provenance limits apply. First, the E3 test metrics were saved by the original Kaggle run in `results/resnet50_finetune/test_metrics.json`; the full-precision accuracy stored there corresponds to 21,758.5 correct out of 25,250 and therefore cannot be reproduced bit-exactly by `src/evaluate.py`, which counts integer predictions. The rounded values (86.17% accuracy, 86.10% macro-F1) are the ones reported. Second, the saved E1 history records a constant learning rate (E1 is configured without a scheduler), and the saved E3 history also records a constant 10^{-4} rate even though `pretrained.yaml` configures cosine annealing; the exported Kaggle run confirms the E3 cosine schedule was applied, and the logged values are not adjusted.

4.4 Project Artifacts

- Assignment page: `https://tapstoptap.github.io/deep-learning-261/assignment2`
- Source code: `https://github.com/tapstoptap/deep-learning-261/tree/main/assignment2`
- Dataset proposal: `assignment2/proposal/dataset_proposal.md`
- Checkpoints and reconstruction: `assignment2/CHECKPOINTS.md`
- Interactive demo (Streamlit): `assignment2/demo/`, live at `https://deep-learning-261-ahricdx9pw4yfwaqmrmx7e.streamlit.app/`

- Report source and PDF: `assignment2/report/`

5 Contributions and AI Usage Disclosure

5.1 Member Contributions

Member	ID	Assignment 2 contribution	Other assignments
Nguyễn Minh Phúc	2453005	Repository skeleton; Food-101 reference and split check.	Did not contribute to A1.
Nguyễn Thanh Thế	2453191	Dataset proposal; implementation; experiments; figures; report.	Did not contribute to A1.
Nguyễn Minh Quang	2453052	Report review and formatting.	Sole author of A1.

5.2 AI Usage Disclosure

Google Antigravity and Qoder were used at several stages of Assignment 2 for implementation support, result curation, figure generation, report preparation, and a pre-submission audit. The table below records the relevant uses. AI assistance was a support for the work: the group reviewed the suggestions and checked the resulting code, analysis, and reported values against the course handbook, repository artifacts, tests, and saved experiment outputs. All models were trained, logged, and evaluated by the group on Kaggle and Colab GPU instances; no reported metric was generated by a language model.

Table 11. AI assistance used during Assignment 2. Entries correspond to rows 5–7 and 9 of `AI_USAGE.md`.

No.	Member / stage	Purpose and affected sections	Assistance and verification
1	Nguyễn Thanh Thế Implementation 20 Oct 2026	Implemented the baseline CNN and the frozen and fine-tuned ResNet-50 pipelines, configs, and unit tests; affected `assignment2/src/`, `configs/`, `tests/`, and `README.md`.	Helped with the SimpleCNN and ResNet-50 wrappers, training and evaluation scripts, stratified DataLoader, metrics, and tests. Verified locally: six unit tests passed and dummy end-to-end runs completed.
2	Nguyễn Thanh Thế M2 draft 21 Oct 2026	Curated E1/E2 results, training curves, and confusion artifacts for the M2 draft; affected `results/`, `docs/assets/a2/`, and `docs/assignment2.md`.	Parsed training logs, generated curve and confusion figures, and drafted the comparison tables and top-confusion lists; values checked against `history.json` and `test_metrics.json`.
3	Nguyễn Thanh Thế Final experiments 26 Oct 2026	Recorded the E3 fine-tuning run and updated the comparison tables, error analysis, and report draft; affected `results/resnet50_finetune/` and the report sections.	Produced the history, summary, and test-metric files and the training-curve plot; updated the comparison, cost, and limitations sections. Values checked against the Kaggle console output.
4	Nguyễn Thanh Thế Pre-submission audit 4–8 Nov 2026	Audited the M3 package and repaired documentation, figures, and build issues; affected `src/eda.py`, `results/eda/`, `docs/`, both `.tex` sources, `tools/`, and `CHECKPOINTS.md`.	Corrected the culinary grouping to 101/101 classes, removed untraceable confusion counts, replaced unsupported throughput and VRAM figures with measured benchmarks, clarified the protocol confound, added the reconstruction contract and the A0 matched control with its benchmark and qualitative grid, and rebuilt both PDFs. Every number was checked against saved JSON artifacts; six tests passed; no metric was invented.

The complete AI-use log, including prompts, affected files, verification, and the responsible member, is maintained in `AI_USAGE.md`. The final responsibility for the code, experiments, interpretation, and submitted report remains with the group members.

References

[1] L. Bossard, M. Guillaumin, and L. Van Gool. Food-101 – Mining discriminative components with random forests. In *European Conference on Computer Vision (ECCV)*, 2014.

[2] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[3] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.

[4] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[5] S. Kornblith, J. Shlens, and Q. V. Le. Do better ImageNet models transfer better? In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.